



University of Engineering and Technology Lahore

PACMAN GAME

OOP Java | CEP Project Report

Submitted By:	Muhammad Zohaib Shahzad
Registration No:	2025-CYS-167
Semester:	Second Semester
Department:	Department of Computer Engineering
Program:	Cybersecurity
Subject:	Object Oriented Programming - Java
Submitted To:	Mr. Hafiz Muhammad Mubashir Imran

TABLE OF CONTENTS

1. Introduction
 2. Objectives
 3. Game Design
 4. Initial Setup
 5. Control System
 6. Game Logic
 7. Code
 8. Functional Flow
 9. Architectural Diagram
 10. Key Features
 11. Conclusion
-

1. Introduction

This lab report documents the design and implementation of a console-based Pac-Man Adventure game developed using the Java programming language. The game was created as part of an Object-Oriented Programming (OOP) course to demonstrate practical understanding of programming concepts such as class design, object instantiation, encapsulation, user interaction, and file handling.

This game uses the core concept of Java OOP.

The game runs entirely in the terminal, using ANSI escape codes to produce colorful, styled output. The player navigates a Pac-Man character through a 15x15 grid map, collecting dots, avoiding walls and barriers, evading a ghost, and attempting to reach a star (★) to win the game.

The project integrates several core concepts of Java programming and OOP, including class design, conditionals, loops, arrays, file I/O, and randomization—all demonstrated within the context of a fun, interactive game.

2. Objectives

The primary objectives of this lab project are:

- Apply Object-Oriented Programming concepts, specifically class design and object usage through the Ghost class.
- Develop a functional, interactive game using Java's core libraries ([Scanner](#), [FileWriter](#), [Random](#), [ArrayList](#)).
- Design and render a 2D grid map using a 2D String array with ANSI color-coded characters.
- Implement game logic, including movement, collision detection, win/loss conditions, and scoring.
- Demonstrate file handling by persisting game results to a text file (Record.txt).
- Practice difficulty selection mechanics by adjusting the number of player lives based on user choice.
- Utilize ANSI escape codes for enhanced console-based **UI** presentation

3. Game Design

The game is designed around a 15x15 character grid that serves as the game board. The design philosophy focuses on clarity, interactivity, and engagement using only text-based console output.

3.1 Map Layout

The map is constructed from the following visual elements:

Symbol	Color	Meaning
█	Blue	Outer boundary wall — cannot pass through
▣	Red	Inner barrier/obstacle — loses a life on collision
.	Yellow	Collectible dot — increases score by 1
★	Green	Winning goal — reaching it ends the game with a win
@	White	Pac-Man (player character)
□	Purple	Ghost — randomly moving enemy

3.2 Difficulty System

At game start, the player selects a difficulty level, which determines the number of lives:

Difficulty	Lives	Challenge
EASY	5	Forgiving — good for new players
MEDIUM (default)	3	Balanced challenge
HARD	1	One mistake ends the game

4. Initial Setup

The initial setup phase prepares the game environment before any player input is accepted. The following steps occur during initialization:

4.1 Variable Declarations

Variable	Type	Purpose
arr[][]	String[15][15]	2D grid storing map characters
n, m	int	Player position (row, column) — starts at (2,2)
score	int	Tracks collected dots
lives	int	Remaining lives based on difficulty
move	char	Stores current directional input

4.2 Map Initialization

The map is initialized in two passes. First, every cell is filled with a space character. Then, the map is re-drawn with border walls (■), inner barriers (■), collectible dots (·), and the star goal (★) at position [12][12]. The Ghost is placed at [7][7] and the player at [2][2].

4.3 Stylish Banner

A large ASCII art banner is displayed at startup using ANSI yellow bold coloring, followed by the difficulty selection menu styled in cyan with box-drawing characters.

5. Controls System

The player controls Pac-Man using keyboard input via Java's Scanner class. The movement system reads a single character per turn:

Key	Direction	Grid Effect
W	Up	Decrements row index (n--)
S	Down	Increments row index (n++)
A	Left	Decrements column index (m--)
D	Right	Increments column index (m++)

The input is converted to lowercase before processing to handle both uppercase and lowercase entries. Any character other than w/a/s/d is rejected with an error message, and the player is prompted to enter again. This prevents invalid positions and crash scenarios.

6. Game Logic

The core game logic runs within an infinite while(true) loop that continues until a win or loss condition is met.

6.1 Movement & Collision

Before applying the move, the previous position (prevN, prevM) is saved. After updating the position, the cell content at the new coordinates is checked:

- If the cell contains '■' or '■': movement is reversed to the previous position and one life is deducted.
- If lives reach 0: Game Over screen is displayed and the loop breaks.
- If the cell contains '★': Win screen is shown and the loop breaks.
- If the cell contains '·': The score is incremented by 1.
- Otherwise: The player moves and the board is redrawn.


```
System.out.println("\u001B[0m");

System.out.print("Enter Your Name : ");
String name = sc.next() ;

// — Difficulty Level —
System.out.println("\u001B[36m\u001B[1m");
System.out.println("    ╰═══════════════════╯");
System.out.println("    │      CHOOSE DIFFICULTY      │");
System.out.println("    ╰═══════════════════╯");
System.out.println("    │   1. EASY     (5 lives)   │");
System.out.println("    │   2. MEDIUM  (3 lives)   │");
System.out.println("    │   3. HARD     (1 life)    │");
System.out.println("    ╰═══════════════════╯\u001B[0m");
System.out.print(" Choice (1/2/3): ");
int choice = 2;
try { choice = Integer.parseInt(sc.next().trim()); } catch(Exception e){}

int lives;
String LEVELName;
if (choice == 1)        { lives = 5; LEVELName = "EASY"; }
else if (choice == 3) { lives = 1; LEVELName = "HARD"; }
else                    { lives = 3; LEVELName = "MEDIUM"; }

System.out.println("\u001B[32m" + name + " You have " + lives + "
lives." + "\u001B[0m");

//          Storage Of Map
for (x = 0 ; x<15 ; x++){
    for (y = 0 ; y<15 ; y++){
        System.out.print(arr[x][y]=" ");
    }
}

//          Designing The Map
for (x = 0 ; x < 15 ; x++){
    for (y = 0 ; y < 15 ; y++){
        if (x==0 || x==14){
            arr[x][y]='\u001B[34m' +"█" ;
        }
        else if (y==0 || y==14){
            arr[x][y]= "\u001B[34m" +"█" ;
        }
    }
}
```

```

    }
    else if ( x == 5 && y > 4 && y < 10){
        arr[x][y] = "\u001B[31m" + "█";
    }
    else if ( x == 10 && y > 1 && y < 10){
        arr[x][y] = "\u001B[31m" + "█";
    }
    else if ( x == 8 && y > 6 && y < 14){
        arr[x][y] = "\u001B[31m" + "█";
    }
    else if ( y == 5 && x > 8 && x < 14) {
        arr[x][y] = "\u001B[31m" + "█";
    }
    else if ( y == 3 && x > 4 && x < 9) {
        arr[x][y] = "\u001B[31m" + "█";
    }
    else {
        arr[12][12] = "\u001B[32m\u001B[1m" + "★\u001B[0m";
        arr [x][y] = "\u001B[33m" + " .";
    }
}

// printing the map

Ghost ghost = new Ghost(7, 7, arr);
arr[7][7] = "\u001B[35m\u001B[0m";

arr[n][m] = "\u001B[0m" + "@" ;

// Stylish board print
System.out.println("\u001B[34m\u001B[1m
=====");
System.out.println("    ||          PAC-MAN  ADVENTURE          ||");
System.out.println("    ||=====||\u001B[0m");
System.out.print("\u001B[32m    ||  Lives:  ");
for(int i=0;i<lives;i++)
    System.out.print("♥ ");
System.out.println("\u001B[32m          ||\u001B[0m");
System.out.println("\u001B[33m    ||  Score:  " + score + "
\u001B[0m");
System.out.println("\u001B[34m\u001B[1m
=====||\u001B[0m");
for (x = 0 ; x<15 ; x++){

```



```

        System.out.print("\u001B[34m\u001B[1m  || \u001B[0m");
        for (y = 0 ; y<15 ; y++){
            System.out.print(arr[x][y]+" ");
        }
        System.out.println("\u001B[34m\u001B[1m||\u001B[0m");
    }
    System.out.println("\u001B[34m\u001B[1m
    _____\u001B[0m");

//      handling packman
    while(true){
        System.out.println("\u001B[32m" + name + " !  Enter Your Move
(w/a/s/d) :");
        move = sc.next().toLowerCase().charAt(0) ;

        int prevN = n, prevM = m;

        if(move == 's'){
            n++ ;
        }
        else if(move == 'w'){
            n-- ;
        }
        else if(move == 'a'){
            m-- ;
        }
        else if(move == 'd'){
            m++ ;
        }else {
            System.out.println("Hi " + name  +" You Entered Invalid Input
!");
            continue;
        }

//      over condition
        if (arr[n][m].contains("█") || arr[n][m].contains("█")){
            n = prevN; m = prevM;
            lives--;
            if(lives <= 0){
                for (int i = 0 ; i < 20 ; i++) System.out.println();
                // — Game Over Screen —
                System.out.println("\u001B[31m\u001B[1m");
                System.out.println("
    _____");
            }
        }
    }
}

```

```

        System.out.println(" || ||");
        System.out.println(" || □ GAME OVER □ ||");
        System.out.println(" || ||");
        System.out.println(" ||=====||");
        System.out.printf (" || Player:  %-22s||%n", name);
        System.out.printf (" || Score:   %-22d||%n", score);
        System.out.println(" || ||");
        System.out.println(" || Try Again □ ||");
        System.out.println(" ||=====||\u001B[0m");

        break;
    } else {
        System.out.println("\u001B[31m Ouch! You Have Remaining
Lifes : " + lives + "\u001B[0m");
    }
}

// Winning Condition
else if (arr[n][m].contains("★")) {
    arr[prevN][prevM]=" ";
    arr[n][m] = "\u001B[35m\u001B[1m" + "@";
    for (int i = 0 ; i < 20 ; i++) System.out.println();
    // — Win Screen —
    System.out.println("\u001B[32m\u001B[1m");
    System.out.println(" ||=====||");
    System.out.println(" || ||");
    System.out.println(" || □ Congratulations □ ||");
    System.out.println(" || You WIN ||");
    System.out.println(" || ||");
    System.out.println(" ||=====||");
    System.out.printf (" || Champion:  %-21s||%n", name);
    System.out.printf (" || Score:    %-21d||%n", score);
    System.out.println(" || ||");
    System.out.println(" ||=====||\u001B[0m");

    break ;
}

else {
    if (arr[n][m].contains(".")) {
        score++ ;
    }

    arr[prevN][prevM]=" ";

```

```

arr[n][m] = "\u001B[0m" + "@" ;

ghost.move(arr);

if (ghost.catchesPacman(n, m)) {
    lives--;
    System.out.println("☠ Ghost Cought it ! Remaining Lives: " +
lives);

    n = prevN; m = prevM;
}

for (int i = 0 ; i < 20 ; i++) System.out.println();

//      Redraw board
System.out.println("\u001B[34m\u001B[1m
=====");
System.out.println("      ||          PAC-MAN   ADVENTURE          ||");
System.out.println("=====||\u001B[0m");
System.out.print("\u001B[32m      ||   Lives: ");
for(int i=0;i<lives;i++) System.out.print("♥ ");
System.out.println("\u001B[32m          ||\u001B[0m");
System.out.println("\u001B[33m      ||   Score: " + score + "
\u001B[0m");

System.out.println("\u001B[34m\u001B[1m
=====||\u001B[0m");

for (x = 0 ; x<15 ; x++){
    System.out.print("\u001B[34m\u001B[1m      ||  \u001B[0m");
    for (y = 0 ; y<15 ; y++){
        System.out.print(arr[x][y]+" ");
    }
    System.out.println("\u001B[34m\u001B[1m||\u001B[0m");
}

System.out.println("\u001B[34m\u001B[1m
=====||\u001B[0m");

}

}

try {
    FileWriter file = new FileWriter("Record.txt");
    file.write(name + " | " + LEVELName + " | " + score );
    file.close();
}

```

```

System.out.println(".....Entered.....");
    } catch (IOException e) {
        System.out.println("Error: " + e);
    }
}
}

```

7.2 Ghost.java

```

import java.util.*;
class Ghost {
    int row, col;
    String prevCell;
    Random rand = new Random();

    Ghost(int startRow, int startCol, String[][] arr) {
        this.row = startRow;
        this.col = startCol;
        this.prevCell = arr[startRow][startCol];    }

    void move(String[][] arr) {
        int[] dRow = {-1, 1, 0, 0};
        int[] dCol = {0, 0, -1, 1};

        arr[row][col] = prevCell;

        List<Integer> dirs = new ArrayList<>(Arrays.asList(0, 1, 2, 3));
        Collections.shuffle(dirs, rand);

        for (int d : dirs) {
            int newRow = row + dRow[d];
            int newCol = col + dCol[d];

            if (!arr[newRow][newCol].contains("■") &&
                !arr[newRow][newCol].contains("█")) {
                prevCell = arr[newRow][newCol]; //
                row = newRow;
                col = newCol;
                break;
            }
        }
    }
}

```

```

    }

    }

    arr[row][col] = "\u001B[35m\u001B[0m";    }
    boolean catchesPacman(int pRow, int pCol) {
        return row == pRow && col == pCol;
    }
}

```

8. Functional Flow

The program follows a clear sequential flow from startup to game conclusion:

Step	Action	Outcome
1	Display banner & prompt name	Player identity stored
2	Show difficulty menu	Lives set (1 / 3 / 5)
3	Initialize 15x15 map array	Map ready with walls, dots, star
4	Place Ghost at [7][7], Player at [2][2]	Game objects positioned
5	Display initial board	Player sees the map
6	Read move (w/a/s/d)	Player direction captured
7	Check collision / win / dot	Lives or score updated
8	Move ghost randomly	Ghost changes position
9	Check ghost-player collision	Life deducted if caught
10	Redraw board	Updated state displayed
11	Repeat steps 6-10	Loop continues
12	Win or Game Over triggered	Final screen shown
13	Write to Record.txt	Score persisted to file

10. Key Features

Feature	Description	Java Concept Used
Colorful Console UI	ANSI escape codes for colored text and styled panels	String manipulation
Dynamic Difficulty	3 levels with different life counts	Conditionals, user input
2D Grid Map	15x15 String array with visual characters	2D Arrays
Ghost AI	Random walk avoiding walls using shuffle	ArrayList, Collections, Random
Score System	Dot collection increments score	Variables, conditionals
Collision Detection	Wall, barrier, ghost, and star checks	String.contains(), logic
File Persistence	Saves name, level, and score to Record.txt	FileWriter, IOException
Win/Loss Screens	Formatted end-game screens with player stats	printf formatting

11. Conclusion

This project successfully demonstrates the implementation of a console-based Pac-Man Adventure game using Java. The program integrates multiple foundational programming concepts — 2D arrays, loops, conditionals, object-oriented class design, file I/O, and randomization — in a cohesive and functional application.

The Ghost class showcases basic OOP principles: encapsulation of state (position, previous cell) and behavior (move, detect collision). The main game loop demonstrates structured problem-solving—handling input, updating state, checking conditions, and rendering output repeatedly until a terminal state is reached.

The difficulty system, score tracking, ANSI-styled interface, and file-based result persistence add practical features that go beyond a minimal implementation, making this project a well-rounded demonstration of Java programming skills.